Prise en main de l'outil Digital

Pour comprendre comment fonctionnent les circuits numériques, nous allons non seulement les dessiner mais observer leur fonctionnement grâce à un simulateur appelé *Digital* (`https://github.com/hneemann/Digital`).

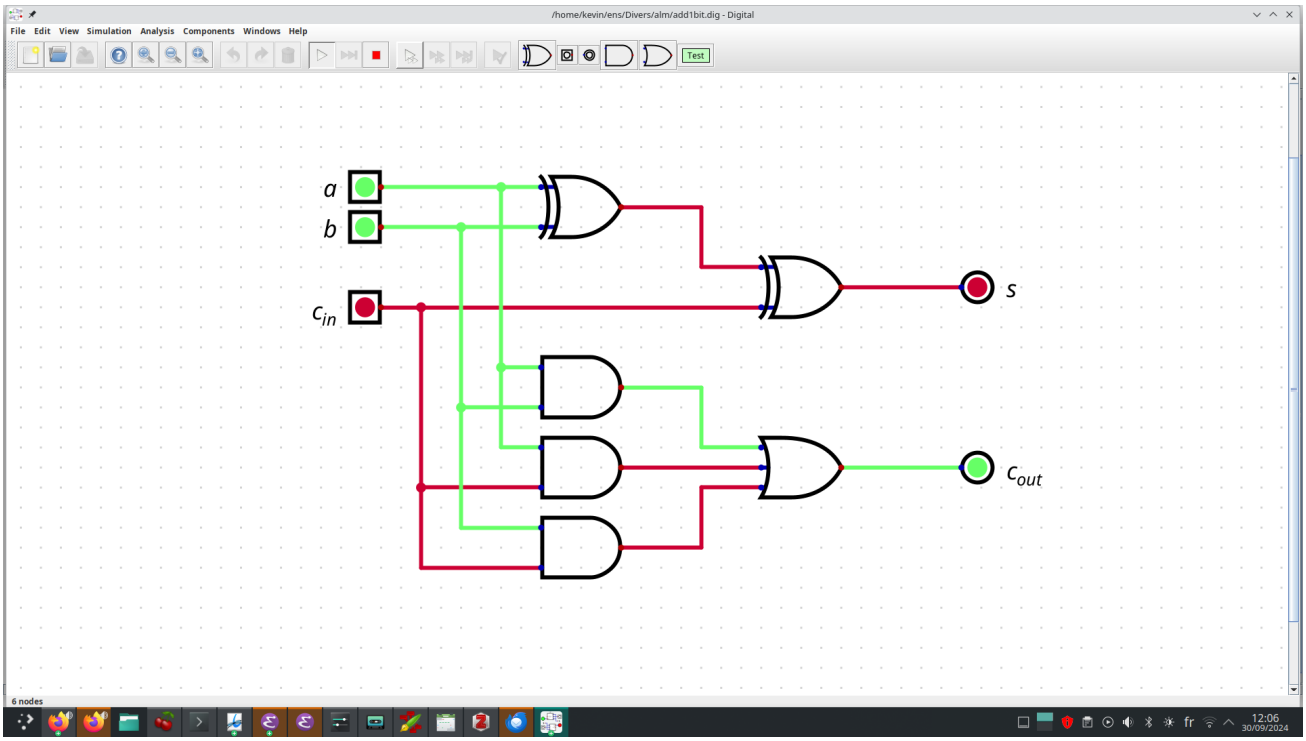


FIGURE 1 – Exemple de simulation de circuit grâce à Digital

Grâce à ce logiciel, une fois que l'on a dessiné un composant, il est possible d'observer les valeurs portées par les fils et les portes logiques, comme si les composants électroniques existaient. Cela peut être pratique dans votre cas pour simuler les composants que vous allez dessiner pendant le reste du semestre et que vous ne comprendriez pas bien. Par exemple, vous pourrez, lors d'une prochaine séance, simuler l'exécution d'un processeur !

Mais pour le moment, il faut apprendre comment se servir du logiciel. C'est l'objet de ce sujet, qui vous permettra tout de même de réaliser et simuler un circuit implémentant une addition sur 8 bits, avec retenue.

1 Premières manipulations

Pour lancer Digital, effectuez ces quelques étapes :

— Dans un terminal, créez un répertoire `TP_Digital` et déplacez-vous dedans.
— Télécharger le fichier `Digital.jar`, disponible sur Moodle et placez-le dans le répertoire sus-mentionné.
— Exécutez la commande : `java -jar Digital.jar` dans un terminal. Le logiciel devrait démarrer.

Commencez par passer à l'affichage classique des portes logiques (au lieu d'une version simplifiée) via le menu `Edit→Settings→Use IEEE 91-1984 shapes` (cochez la case).

Maintenant, suivez le tutoriel de la documentation, accessible depuis le menu `View→Start Tutorial`. Si besoin, n'hésitez pas à aller voir la documentation, accessible sur Moodle également.

Enfin, afin de finir de vous familiariser avec les fonctionnalités basiques, testez ces diverses petites manipulations :

— **pour zoomer/dézoomer** : CTRL-<molette>
— **pour déplacer un composant** : cliquez dessus (gauche) puis bouger la souris. Il faut ensuite retirer les fils... C'est vraisemblablement l'aspect le plus agaçant de ce logiciel, mais ne vous découragez pas pour cela.
— **pour supprimer un composant** : CTRL-<clic gauche> puis <suppr>
— **pour supprimer un fil** : CTRL-<clic gauche> puis <suppr>
— **pour sélectionner plusieurs fils ou composants** : <clic gauche> en dehors de tout composant ou fil puis entourer les composants que vous voulez bouger ou supprimer.
— **pour accéder aux propriétés d'un composants** : <clic droit> sur un composant. À quoi cela sert-il ? Eh bien ar exmple, par défaut les portes ont 2 entrées. Si vous avez besoin d'un AND ou d'un OR à trois entrées (ou plus) vous pouvez changer le nombre d'entrées. Vous pouvez aussi changer l'orientation des portes, complémenter certaines entrées des portes, et autres.
    — par exemple, **pour donner un nom à une entrée** : <clic droit> sur cette entrée, puis modification du nom (label en anglais).

Pour finir cette prise en main, donnez un nom aux deux entrées, et à la sortie.

Voilà, vous pouvez maintenant passer à la réalisation d'un additionneur 1 bit !

2 Additionneur 1 bit

Créez un nouveau circuit (`File->New Embedded Circuit`). Les fichiers fournis avec le sujet ne contiennent que les entrées, les sorties et le bloc test qui sera utile pour les tests. Vous pouvez démarrer depuis ce fichier ou créer le votre et copier le bloc test.

Vous allez réaliser un additionneur 1 bit. Ce circuit possède trois entrées `a`, `b` et `c_in` représentant respectivement les deux valeurs à additionner et la retenue entrante de l'addition. Il possède deux sorties :

— `s`, dénotant la valeur de la somme
— et `c_out`, dénotant la valeur de la retenue de sortie.

Plus précisément, voici la définition de s et `c_out` :

— `s = (a xor b) xor c_in`
— `c_out = (a and b) or (a and c_in) or (b and c_in)`

Sauvegardez ce circuit dans un fichier nommé `add1bit.dig`. D'une manière générale, utilisez toujours des noms informatifs car vous allez ensuite pouvoir réutiliser vos circuits pour en construire d'autres, plus complexes.

Maintenant, simulez pas-à-pas votre additionneur 1 bit :

— démarrez la simulation en cliquant sur l'icône avec le triangle
— simulez l'addition de 1 + 1 sans retenue entrante (`a` et `b` = 1). La sortie `s` vaut-elle bien 0 ? La retenue sortante est-elle correcte ?
— passez à 1 la retenue entrante `c_in`. La sortie s est-elle correcte ?
— Ouvrez la fenêtre d'affichage des mesures (`Menu Simulation ->Show measurement graph`). Combien y a-t-il de combinaison avec 3 entrées ? Continuez à modifier `a`, `b`, et `c_in` dans votre circuit jusqu'à avoir observé toutes les combinaisons de valeurs d'entrée.

Enfin, via le menu `Analysis→Analysis`, demandez à Digital de calculer la table de vérité de votre circuit. Vérifiez que vous êtes d'accord avec cette table de vérité, puis passez à l'exercice suivant.

3 Additionneur 8 bits

Créez un nouveau circuit (`File->New Embedded Circuit`).

Vous allez maintenant construire un additionneur 8 bits. Pour cela, vous allez utiliser 8 instances du composant « additionneur 1 bits » que vous venez d'implémenter.

Lisez la partie 1.4 de la documentation (`Hierarchical design`). Créez un nouveau fichier Digital avec `File→New Embedded Circuit`. Sauvegardez-le tout de suite sous le nom `add8bits` dans le même répertoire que votre `add1bit`. Ce dernier devrait apparaître dans la liste des composants disponibles, sous la rubrique `Components→Custom`.

Maintenant, effectuez 8 copies de ce composant `add1bit` et connectez-les pour réaliser un circuit additionneur 8 bits, sans vous préoccuper des entrées et sorties pour le moment. Placez vos composants de manière lisible ; d'ailleurs notez que Digital vous permet de choisir plusieurs dispositions pour les entrées-sorties d'un circuit, en cliquant (à droite) sur un composant, puis menu déroulant `shape`.

Enfin, il va falloir vous occuper des entrées et des sorties de votre additionneur. Pour cela :

— Placer les entrées (`menu Composants->inputs`) et sorties (`Composants->outputs`).
— Nommez ces entrées et sorties de manière pertinente : `a`, `b`, `c_in` pour les entrée, `s`, `c_out` pour le sorties.
— Combien de bits constitue chaque entrée et chaque sortie ? Modifiez leur taille en cliquant droit sur chacune d'elle et en modifiant leur nombre de bits.
— Afin de relier chaque bit des entrées/sorties à un composant différent, il vous faut utiliser des composants `Splitter/Merger` (menu `Components→Wires`). Dans les propriétés de ce composant, vous pouvez en effet préciser le nombre de bits en entrée et en sortie. Par exemple, pour extraire le $3^{\text{ème}}$ de 8 bits, vous indiquez `8` dans `Input Splitting` et `3-3` dans `Output Splitting`.

Pensez à sauvegarder votre circuit. Nous allons maintenant passez à la phase de simulation :

— **Sur papier**, effectuez l'opération 0xA4 + 0x5B en binaire.
— **Simulez** l'opération précédente. Le résultat est-il correct ?
— Positionnez à 1 la retenue entrante. Le résultat est-il toujours correct ?

Bravo ! Vous êtes maintenant initiés à la simulation de circuits électronique. Vous pouvez maintenant consolider vos connaissances en créant un comparateur (cf. suite du sujet) ou n'importe quel circuit que vous voudriez tester pour mieux le comprendre.

4 Unité Arithmétique et Logique (UAL)

Dans les sections précédentes, vous avez eu une première expérience assez guidée pour prendre en main l'outil, créer un circuit et le simuler. Dans cette partie, nous vous donnons le cahier de charge d'un circuit à réaliser à partir de l'additionneur 8 bits et des composants déjà inclus dans *Digital* et c'est à vous de créer le circuit. Il existe plusieurs solutions pour réaliser le circuit. Votre objectif est de créer un circuit qui valide le cahier de charge. Ne vous inquiétez pas pour l'instant si vous pensez qu'il y a des optimisations possibles.

Dans le sujet, vous trouverez `ual.dig`, un fichier qui contient les entrées et les sorties attendues [1]. Plus important encore, vous trouverez les blocs de tests pour valider votre circuit. Il est possible de désactiver ou d'activer une partie des tests pour tester un sous ensemble de votre circuit.

Rappel : il est conseillé d'utiliser des multiplexeurs (ou des démultiplexeurs). Il est aussi conseillé *de tester au fur et à mesure.*

L'UAL a 3 entrées et une sortie :

— `a` : Une entrée 8 bits pour le premier opérande
— `b` : Une entrée 8 bits pour le second opérande
— `op` : Une entrée 3 bits pour sélectionner l'opération à effectuer
— `s` : Une sortie 8 bits pour le résultat de l'opération
— *(Optionnel)* 4 sorties de 1 bit pour chacun des flags Z, C, V et N.

L'opération à effectuer dépend de `op` :

— 000 (addition) : $s = a + b$.
— 001 (OU logique) : $s = a|b$.
— 010 (ET logique) : $s = a\&b$.
— 011 (shift logique) : $s = a << b$.
— 100 (soustraction) : $s = a - b$.
— XXX (autre) : indéfini [2]

Vous utiliserez l'additionneur 8 bits créé ci-dessus. Le reste des composants (à l'exception de la soustraction, détaillée ci-dessous) sont disponibles dans `Components`.

1. Notion de **banc de test**
2. Nous n'allons pas vérifier les cas où *op* prend des valeurs indéfinies. Par contre il est important que votre circuit *fasse quelque chose*. Avoir une sortie nulle suffit.

4.1 Soustraction

Implémenter la soustraction est possible avec l'additionneur 8 bits, avec quelques modifications simples. En notant qu'une soustraction $a - b = a + (-b)$, le défi revient à trouver le complément à 2 de b.

Astuce : $a + (-b) = a + \bar{b} + 1$. Une entrée supplémentaire peut être branchée à `c_in` du premier additionneur. Vous pouvez inverser les bits au niveau de l'UAL.

4.2 Validation

C'est possible de commencer avec des simulations manuelles pour comprendre le fonctionnement de votre circuit. Pour tester plus rapidement et détecter les régressions [3], il est plus pertinent d'utiliser des blocs de tests.

Faites tourner les tests plusieurs fois. Est-ce que les valeurs changent ? Inspecter le contenu des tests. Est-il pertinent de tester de cette manière ? Est-il suffisant ? Est-ce que c'est faisable de tester exhaustivement ce circuit ? Et s'il s'agissait d'un UAL 64 bits ?

4.3 (Optionnel) Drapeaux de l'UAL

Si vous avez encore le temps (ou bien vous voulez aller plus loin), il est assez direct d'ajouter les drapeaux Z, C, V et N. Les drapeaux ne sont pas nécessairement valables pour chaque opération. Par exemple, il n'y a pas de sens de vérifier le drapeau C suite à une opération d'un ET logique. Heureusement, l'interprétation des sorties n'est pas l'objectif de ce circuit.

Autrement dit, votre implémentation doit être correcte quand l'opération a du sens. Vous avez le choix de définir ce qui vous arrange lorsque le drapeau n'est pas sensé être utilisé.

Pour faire cela, créez les sorties pour chaque drapeau. *Rappel* : voici la définition de chaque drapeau :

- Z (Zero) : tous les bits de la sortie sont nuls.
- C (Carry) : valeur de la retenue de l'additionneur de poids le plus fort (le $8^{\text{ème}}$).
- N (Negative) : le bit de poids fort du résultat apparent.
- V (oVerflow) : ce drapeau est mis à 1 si les deux dernières retenues sont différentes [4], qui correspondent à la retenue de sortie `c_out` du $7^{\text{ème}}$ et $8^{\text{ème}}$ additionneurs. Vous devriez alors modifier l'additionneur 8 bits en ajoutant une sortie supplémentaire de l'avant dernière retenue.

Vérifiez votre circuit, de préférence avec un bloc test avec des cas d'exemples.

5 (Optionnel) Comparateur

Maintenant, réalisez un circuit combinatoire testant l'égalité entre 2 mots binaires de 8 bits. La sortie est un bit valant 1 en cas d'égalité, 0 sinon.

Testez votre comparateur et sauvegardez-le.

3. https://fr.wikipedia.org/wiki/R%C3%A9gression_logicielle, même si on parle ici de la régression dans la conception des circuits numériques. Les concepts ne changent pas.
4. Pensez à utiliser un `XOR`